

TECH

JAVASCRIPT FUNDAMENTALS

EVENT LOOP / ASYNC CLOSURES / SCOPE / MEMORY **PROTOTYPES / OBJECTS / CLASSES**

PROTOTYPES / OBJECTS / CLASSES

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК learn.javascript.ru

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	ПРОТОТИПЫ new Object.create / as sign / freeze Q1 · Q5 · Q9	База прототипного наследования и инструменты работы с объектом. Без этого нельзя честно говорить про классы.
ДЕНЬ 2 ТЯЖЁЛЫЙ	this ARROW vs REGULAR call / apply / bind Q2 · Q3 · Q4	Самая частая боль на собеседовании. Понимание this — половина вопросов про функции и методы.
ДЕНЬ 3 ТЯЖЁЛЫЙ	КЛАССЫ В JS PRIVATE FIELDS Q6 · Q20	Чем JS class отличается от классических ООП-языков, и как теперь делают приватные поля.
ДЕНЬ 4 СРЕДНИЙ	PROPERTY DESCRIPTORS ENUMERABLE Q7 · Q8	Свойства объекта изнутри: что у них за флаги, и как это всплывает в for-in / Object.keys.
ДЕНЬ 5 СРЕДНИЙ	ПРИБЛИЖЕНИЕ COERCION NUMBER / NaN / Infinity Q10 · Q11 · Q12	Как JS приводит типы и сравнивает значения. Куча edge-cases — спрашивают почти всегда.
ДЕНЬ 6 ЛЁГКИЙ	OPTIONAL CHAINING NULLISH vs ?? Q13 · Q14 · Q15	Современные операторы — синтаксический сахар, но детали важны (?? только null/undefined, не falsy).
ДЕНЬ 7 СРЕДНИЙ	ITERABLE / ITERATOR GENERATORS S SYMBOLS Q16 · Q17 · Q18 · Q19	Закрепление: как устроены for-of, генераторы, что такое Symbol и где он реально нужен.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

ПРОТОТИПЫ `new` `Object.create` / `assign` / `freeze`

ЧТО УЧИМ В ЭТОТ ДЕНЬ

База прототипного наследования и инструменты работы с объектом. Без этого нельзя честно говорить про классы.

ВОПРОСЫ ДНЯ

Q01 Что такое prototype chain?

Q05 Как работает new?

Q09 Что делают Object.create, Object.assign, Object.freeze?

Что такое prototype chain?

ОПИСАНИЕ

Prototype chain (цепочка прототипов) — это механизм наследования в JS. Каждый объект имеет внутреннюю ссылку `[[Prototype]]` на другой объект (либо `null`). Когда читаешь свойство, движок сначала ищет его в самом объекте; если не нашёл — идёт в `[[Prototype]]`; не нашёл там — идёт дальше, до конца цепочки. Объекты, созданные литералом `{}`, наследуют от `Object.prototype`; массивы — от `Array.prototype`, который сам наследует от `Object.prototype`. Цепочка заканчивается на `null`.

КАК РАБОТАЕТ

- 01 У каждого объекта есть скрытое поле `[[Prototype]]`. Получить его можно через `Object.getPrototypeOf(obj)`, установить — `Object.setPrototypeOf(obj, proto)` (медленно, лучше избегать).
- 02 Литералы: `{}` наследует `Object.prototype`; `[]` — `Array.prototype`; `function() {}` — `Function.prototype`; `class C {}` с `new` — `C.prototype`.
- 03 Поиск свойства идёт по цепочке снизу вверх: сначала сам объект, потом `[[Prototype]]`, потом `[[Prototype]]` прототипа, и так до `null`.
- 04 Запись свойства через `obj.x = ...` ВСЕГДА пишет в сам объект, не в прототип (за исключением сеттеров).
- 05 Свойство `__proto__` — устаревший геттер/сеттер для `[[Prototype]]`. В современном коде используют `Object.getPrototypeOf` / `setPrototypeOf` или `Object.create`.
- 06 `for-in` перечисляет свойства самого объекта и его прототипов (только `enumerable`). `Object.keys` / `.values` / `.entries` — только собственные.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Используется во всех ООП-конструкциях JS — `class`, `new`, наследование.
- 02 Полиморфизм методов: метод объявлен на прототипе, переопределить можно в потомке.
- 03 Расширение встроенных объектов через `prototype` (например, `Array.prototype.myMethod`). Делать осторожно — может конфликтовать с будущим стандартом.
- 04 `Object.create(null)` — создание объекта без прототипа, удобно для словарей без конфликта с `toString` и т.д.

ПОДВОДНЫЕ КАМНИ

- 01 `Object.setPrototypeOf` и `__proto__ = ...` — медленные операции, ломают оптимизации движка (`hidden classes`).
- 02 Расширение `Object.prototype` — почти никогда не делается, потому что добавит свойство ко ВСЕМ объектам и сломает `for-in` в чужом коде.
- 03 `for-in` перечисляет наследованные свойства — обычно нужен `Object.hasOwn(obj, key)` или `Object.keys`.

04 Чёрные дыры: `Object.create(null)` – у такого объекта нет `.toString`, `.hasOwnProperty` и т.д. Это и плюс (чистый словарь), и сюрприз.

05 `instanceof` проверяет цепочку прототипов: а `instanceof B = true` если `B.prototype` есть в `[[Prototype]]`-цепочке а.

КОД

// Цепочка для литералов

```
const o = {};  
Object.getPrototypeOf(o) === Object.prototype; // true  
Object.getPrototypeOf(Object.prototype) === null; // true
```

// Поиск метода – поднимается по цепочке

```
const arr = [1, 2, 3];  
arr.map(x => x); // map нет в самом arr  
// нашёлся в Array.prototype.map
```

// `Object.create` – явное прототипирование

```
const animal = { eats: true };  
const rabbit = Object.create(animal);  
rabbit.jumps = true;  
rabbit.eats; // true (из animal)  
rabbit.jumps; // true (своё)
```

// `instanceof` по цепочке

```
class A {}  
class B extends A {}  
const b = new B();  
b instanceof B; // true  
b instanceof A; // true
```

МОГУТ ДОСПРОСИТЬ

01 Чем `Object.create` отличается от `new`?

02 Что такое `hidden class` и почему `setPrototypeOf` его ломает?

03 Как правильно перебрать только собственные свойства?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/prototype-inheritance>

Как работает new?

ОПИСАНИЕ

Оператор `new` при вызове функции делает четыре вещи: создаёт новый пустой объект, привязывает его `[[Prototype]]` к `F.prototype` вызываемой функции, вызывает функцию с `this` = этот новый объект, и возвращает либо то, что вернула функция (если это объект), либо сам новый объект (если функция вернула примитив или ничего). Ключевое – связь с `prototype`: `new` ставит правильную цепочку прототипов, и поэтому метод, положенный на `F.prototype`, доступен у всех `instance`-ов через `[[Prototype]]`.

КАК РАБОТАЕТ

- 01 Шаг 1: создать новый объект `{}` с `[[Prototype]] = F.prototype` (если `F.prototype` не объект – берётся `Object.prototype`).
- 02 Шаг 2: вызвать `F` с `this` = этот новый объект и переданными аргументами.
- 03 Шаг 3: если `F` вернула объект – возвращается он; иначе – новый объект (примитивы и `undefined` игнорируются как `return value`).
- 04 В строгом режиме внутри `F`-функции `this` – этот новый объект; в нестрогом – то же самое (в `new` всегда). Если `F` вызвать без `new` в нестроге – `this = window/global`, в строгом – `undefined`.
- 05 ES2015+ есть встроенная защита: `new.target` внутри функции позволяет проверить, вызвана ли она через `new`. Если без `new` – `new.target === undefined`.
- 06 Стрелочные функции и большинство встроенных вроде `Math` не могут быть конструкторами – у них нет `[[Construct]]`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая работа с классами – `class` под капотом использует `new`.
- 02 Конструкторы дляDDD-моделей, фабрики объектов с методами на прототипе.
- 03 Расширение встроенных типов: `new Map()`, `new Set()`, `new Date()`.
- 04 Pattern «защита от вызова без `new`»: `if (!new.target) throw new TypeError('use new')`.

ПОДВОДНЫЕ КАМНИ

- 01 Если `F` вернёт объект, он подменит автоматически созданный – `this` новый объект в этом случае «теряется». Иногда это нужно (паттерн `private state`), но обычно – ловушка.
- 02 Стрелочные функции в качестве конструкторов – `TypeError`. Это `by design`: у них нет своего `this` и `[[Construct]]`.
- 03 `F.prototype = ...` после того, как уже сделан `new F()` – уже созданные `instance`-ы продолжают ссылаться на старый `prototype`.
- 04 Вызов конструктора без `new` – частый баг: `this` – глобальный объект (или `undefined` в `strict`). Защищаются через `new.target`.

05 Если переопределить `F.prototype = { ... }` целиком, `constructor` по умолчанию теряется. Нужно вернуть руками `F.prototype.constructor = F`.

КОД

// Что делает new – псевдокод

```
function NEW(F, ...args) {  
  const obj = Object.create(F.prototype);  
  const result = F.apply(obj, args);  
  return (result && typeof result === 'object') ? result : obj;  
}
```

// Связь instance ↔ prototype

```
function User(name) { this.name = name; }  
User.prototype.greet = function () { return 'Hi ' + this.name; };  
const u = new User('A');  
u.greet(); // 'Hi A'
```

// Защита через new.target

```
function Strict() {  
  if (!new.target) throw new TypeError('use new');  
  this.ok = true;  
}  
// Strict() // TypeError  
new Strict(); // ok
```

// Возврат объекта подменяет результат

```
function F() {  
  this.x = 1;  
  return { y: 2 };  
}  
const o = new F();  
o.x; // undefined  
o.y; // 2
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт `new F()`, если `F` – стрелочная функция?
- 02 Что произойдёт, если `F` вернёт строку?
- 03 Зачем вообще `new.target`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/constructor-new>

Что делают `Object.create`, `Object.assign`, `Object.freeze`?

ОПИСАНИЕ

Это три самых частых статических метода `Object` — каждый решает свою задачу. `Object.create(proto, props?)` создаёт новый объект с указанным `[[Prototype]]`. `Object.assign(target, ...sources)` копирует собственные перечислимые свойства источников в `target` и возвращает `target` — это shallow merge. `Object.freeze(obj)` делает объект «замороженным»: запрещает менять, добавлять и удалять свойства. Все три часто всплывают в коде, и их различия любят спрашивать.

КАК РАБОТАЕТ

- 01 `Object.create(proto)`: новый объект с `[[Prototype]] = proto`. Можно передать `null` — объект будет без прототипа.
- 02 `Object.create(proto, descriptors)`: второй аргумент — карта property descriptors, чтобы сразу задать свойства с флагами.
- 03 `Object.assign(target, ...sources)`: обходит каждый source, берёт его собственные перечислимые свойства (включая Symbol-ключи) и пишет в target. Возвращает target.
- 04 `Object.assign` — shallow: вложенные объекты копируются по ссылке, изменение в копии повлияет на оригинал.
- 05 `Object.assign` вызывает геттеры на source и сеттеры на target. Если есть нестандартные descriptor-флаги — теряются.
- 06 `Object.freeze(obj)`: `writable=false`, `configurable=false` для всех свойств. Прототип тоже фиксируется. Shallow — вложенные объекты остаются мутабельными.
- 07 `Object.isFrozen`, `Object.seal` (запрет add/remove, но `writable` не трогает), `Object.preventExtensions` (только запрет add) — более тонкие варианты.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Object.create` — для прототипного наследования без класса, для создания «чистых» словарей через `Object.create(null)`.
- 02 `Object.assign` — простой merge объектов, копирование settings, shallow clone (хотя `{...obj}` читается чище).
- 03 `Object.freeze` — защита config-объекта от случайной мутации, контроль иммутабельности на этапе исполнения.
- 04 В тестах — freeze + проверка, что код не мутирует входные данные.

ПОДВОДНЫЕ КАМНИ

- 01 `Object.create(null)` объекты не имеют `.toString`, `.hasOwnProperty` и т.д. — нужно использовать `Object.hasOwn` / прямые методы `Object`.
- 02 `Object.assign` не копирует non-enumerable свойства, не копирует прототип, не копирует свойства с символьными ключами правильно в старых ES.
- 03 `Object.assign` + геттеры/сеттеры — вызовут их при копировании. Иногда сюрприз.

04 `Object.freeze` – shallow. Глубокая заморозка делается рекурсивно руками или через библиотеку.

05 `Object.freeze` в нестрогом режиме молча игнорирует попытки изменения; в строгом – `TypeError`.

06 `Spread {...a, ...b}` часто проще и читаемее, чем `Object.assign({}, a, b)`.

КОД

// `Object.create` – словарь без прототипа

```
const dict = Object.create(null);
dict.x = 1;
// 'toString' in dict; // false
```

// `Object.assign` – shallow merge

```
const def = { a: 1, b: 2 };
const user = { b: 9, c: 3 };
Object.assign({}, def, user); // {a:1, b:9, c:3}
```

// `Object.freeze` – заморозка верхнего уровня

```
const o = Object.freeze({ x: 1, n: { y: 2 } });
o.x = 9;           // молча в нестрог., TypeError в strict
o.n.y = 9;         // прошло, n не заморожен
```

// Глубокая заморозка

```
function deepFreeze(o) {
  for (const k of Object.keys(o)) {
    const v = o[k];
    if (typeof v === 'object' && v !== null) deepFreeze(v);
  }
  return Object.freeze(o);
}
```

МОГУТ ДОСПРОСИТЬ

01 Чем `Object.assign` отличается от `spread`?

02 Чем `Object.freeze` отличается от `Object.seal` и `preventExtensions`?

03 Что вернёт `Object.create(Object.prototype)`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/native-prototypes>

this ARROW vs REGULAR call / apply / bind

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Самая частая боль на собесе. Понимание `this` – половина вопросов про функции и методы.

ВОПРОСЫ ДНЯ

Q02 Как работает `this`?

Q03 Чем `arrow function` отличается от обычной функции?

Q04 Что делают `call`, `apply`, `bind`?

ВОПРОС Q02

Как работает this?

ОПИСАНИЕ

`this` – специальная привязка функции к объекту-получателю. В отличие от других языков, `this` в JS определяется не местом объявления функции, а способом её вызова. Правил пять: вызов как метод (`obj.f()` → `this = obj`), вызов как обычная функция (`this = undefined` в `strict`, иначе `globalThis`), вызов через `new` (`this =` новый объект), явная привязка через `call/apply/bind`, и стрелочные функции (не имеют своего `this` – наследуют из лексического окружения).

КАК РАБОТАЕТ

- 01 Вызов как метод: `obj.method()` – `this = obj`. Точка перед именем – ключевой признак.
- 02 Вызов как функция: `f()` – в `strict mode` `this = undefined`, в нестрогом – `globalThis` (в браузере `window`).
- 03 Вызов через `new F()` – `this =` новый объект, который вернётся (см. Q5).
- 04 Явная привязка: `f.call(thisArg, ...args)`, `f.apply(thisArg, args)`, `f.bind(thisArg, ...args)`.
- 05 Стрелочные функции – НЕ имеют собственного `this`. Они берут его лексически из окружающей функции/класса.
- 06 Колбэк, переданный методу класса, теряет связь с инстансом: `f = obj.method`; `f()` – `this` уже не `obj`.
- 07 В DOM-обработчиках `this` – это сам элемент (если не стрелка).
- 08 В классах внутри методов `this` – это инстанс. Но если метод вытащить и вызвать отдельно – связь теряется.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой метод объекта/класса работает с `this`. Понимание нужно везде.
- 02 При передаче метода в обработчик события – нужно либо стрелка, либо `bind`.
- 03 В колбэках `setTimeout`, `addEventListener` – частый источник путаницы с `this`.
- 04 В кастомных методах фреймворков (`controllers`, `view-models`) – `this` определяет, чем «сейчас работает» функция.

ПОДВОДНЫЕ КАМНИ

- 01 Главная ловушка: оторвал метод от объекта – потерял `this`. Лечится `bind` или стрелочной обёрткой.
- 02 Стрелочная функция не подходит для методов класса, если важна правильная связка через прототип.
- 03 В `nested`-функции внутри метода `this` уже `undefined/global`. Нужно либо стрелка, либо `self = this`.

04 В классах внутри callback типа `this.items.map(this.process)` – `process` потеряет `this`. Лучше `map(x => this.process(x))` или `bind` в конструкторе.

05 В функциях, вызванных через `new`, `this` – новый объект, независимо от того, что передавали в `bind`.

06 `globalThis` vs `window` vs `self` vs `global` – все похоже, но `globalThis` работает везде стандартно.

КОД

// Метод vs функция

```
const obj = { x: 1, getX() { return this.x; } };
obj.getX();           // 1
const f = obj.getX;
f();                  // undefined (в strict)
                      // или window.x в нестрогом
```

// this теряется в колбэке

```
class Counter {
  count = 0;
  inc() { this.count++; }
}
const c = new Counter();
setTimeout(c.inc, 1000); // this не определён → ошибка
setTimeout(() => c.inc(), 1000); // ок
setTimeout(c.inc.bind(c), 1000); // ок
```

// Стрелка наследует this

```
class Counter {
  count = 0;
  inc = () => { this.count++; }; // стрелка-метод
}
const c = new Counter();
setTimeout(c.inc, 1000); // ок
```

// new – this – новый объект

```
function User(name) { this.name = name; }
const u = new User('A');
u.name; // 'A'
```

МОГУТ ДОСПРОСИТЬ

01 Что вернёт `this` в `strict`-функции, вызванной без контекста?

02 Почему стрелочную функцию нельзя использовать как метод на `prototype`?

03 Что такое `this` в `class-fields`-стрелочных методах?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/object-methods>

ВОПРОС Q03

Чем arrow function отличается от обычной функции?

ОПИСАНИЕ

Стрелочные функции – это укороченный синтаксис, но с важными семантическими отличиями. У них нет своего `this`, `arguments`, `super` и `new.target` – все эти значения берутся из охватывающей лексической среды. Их нельзя вызвать через `new` (нет `[[Construct]]`). У них нет `prototype`-свойства. И они всегда анонимны (если только не присвоены в переменную). Это делает их идеальными для `callbacks` и однострочных функций, но плохими – для методов класса/объекта на `prototype`.

КАК РАБОТАЕТ

- 01 Нет своего `this` – берётся из лексического окружения. Это лечит классическую проблему «потерянного `this`» в колбэках.
- 02 Нет своего `arguments` – нужно использовать `rest`-параметры (`...args`).
- 03 Нет своего `super` – внутри стрелки `super.*` означает то же, что и в окружающем методе.
- 04 Нет своего `new.target`.
- 05 Не могут быть конструкторами – `new arrow() = TypeError`.
- 06 Нет `prototype`-свойства (не путать с `[[Prototype]]`).
- 07 Если возвращают объект-литерал – нужны скобки: `() => ({a: 1})`. Без скобок `{` считается блоком.
- 08 Анонимные по умолчанию. Имя берётся из контекста присваивания: `const f = () => {}`
`→ f.name === 'f'`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Колбэки в `map/filter/reduce/forEach`.
- 02 Колбэки в `.then`-цепочках, обработчиках событий, `setTimeout`.
- 03 Класс-методы как поля для авто-bind: `handle = () => {}`.
- 04 Однострочные функции: `const inc = x => x + 1`.
- 05 В `JSX` – для `inline`-обработчиков (с оговоркой о пересоздании).

ПОДВОДНЫЕ КАМНИ

- 01 Не подходят для методов на `prototype` – потеряют связь с экземпляром. У класса метод вне `fields` идёт в `prototype`, а стрелочная как поле – в экземпляр.
- 02 Не подходят для конструкторов – нельзя вызвать `new`.
- 03 Стрелочный обработчик в `JSX` (`onClick={() => ...}`) пересоздаётся на каждом рендере – память в основном не проблема, но `React.memo` вниз по дереву может сломаться.
- 04 Внутри стрелки `arguments` берётся из ВНЕШНЕЙ функции – иногда сюрприз, потому что выглядит как будто `arguments` должен быть собственный.

05 Стрелка с object-literal без скобок – типичная ошибка: `() => { a: 1 }` возвращает `undefined`, потому что `{ a: 1 }` – это блок с `label`.

КОД

// Стрелка наследует this

```
class Timer {
  start = 0;
  tick = () => { this.start++; }; // this – инстанс
}
```

// Object literal – нужны скобки

```
const make = (x) => ({ value: x });
make(1); // { value: 1 }
```

// Нельзя new

```
const F = () => {};
new F(); // TypeError
```

// arguments – из внешней

```
function outer() {
  const inner = () => arguments[0];
  return inner();
}
outer('hi'); // 'hi'
```

МОГУТ ДОСПРОСИТЬ

- 01 Можно ли стрелочной функции назначить `prototype`?
 - 02 Когда выбрать стрелочный метод-поле, а когда – обычный метод?
 - 03 Что такое лексический `super` и где он встречается?
-

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/arrow-functions>

ВОПРОС Q04

Что делают call, apply, bind?

ОПИСАНИЕ

Все три — методы `Function.prototype`, которые позволяют явно задать `this` и аргументы при вызове функции. `call(thisArg, a, b, c)` — вызывает функцию сразу, передавая аргументы списком. `apply(thisArg, [a, b, c])` — то же, но аргументы массивом. `bind(thisArg, a, b)` — НЕ вызывает, а возвращает новую функцию с зафиксированными `this` и частью аргументов (currying). Используются для явного контроля контекста, частичного применения и заимствования методов.

КАК РАБОТАЕТ

- 01 `call(thisArg, ...args)` — вызывает функцию с `this = thisArg`, передавая остальные аргументы по списку.
- 02 `apply(thisArg, argsArray)` — то же, но аргументы как один массив. Полезно когда массив динамический.
- 03 `bind(thisArg, ...args)` — возвращает новую функцию-обёртку, у которой `this` навсегда зафиксирован = `thisArg`. Можно также зафиксировать первые N аргументов (partial application).
- 04 Bound function игнорирует попытки переопределить `this` через повторный `bind/call/apply` — первая привязка побеждает.
- 05 Bound function — отдельная функция, у неё нет `prototype`, и `instanceof` работает по оригинальной.
- 06 `call` и `apply` — это «вызвать функцию с конкретным `this` и вернуть результат», `bind` — «приготовить функцию для будущего вызова».
- 07 Стрелочные функции игнорируют `thisArg` в `call/apply/bind` — у них нет своего `this`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Заимствование методов: `Array.prototype.slice.call(arguments)` превращает `arguments` в массив. В новом коде есть `[...args]`.
- 02 Частичное применение: `const greetA = greet.bind(null, 'Hi'); greetA('B') === greet('Hi', 'B')`.
- 03 Привязка обработчика к классу: `this.handle = this.handle.bind(this)`.
- 04 Безопасная проверка типов: `Object.prototype.toString.call(x) → '[object Array]'`.
- 05 В некоторых API вызов нужно сделать с конкретным контекстом — `call/apply`.

ПОДВОДНЫЕ КАМНИ

- 01 В старых движках `call/apply` были медленнее обычного вызова. В современных это обычно не имеет значения.
- 02 `bind` создаёт НОВУЮ функцию каждый вызов. В горячем пути (например, в `render` React) `bind` в JSX = переход рекидывается → React.мето вниз ломается.

03 Bound function $\neq$ оригиналу по идентичности – `addEventListener('click', f.bind(this))` и потом `removeEventListener(... f.bind(this))` не сработает (разные функции).

04 Стрелочные функции игнорируют все три. `apply([], [])` над стрелкой – выполнит её с лексическим `this`.

05 В нестрогом режиме `call/apply` с `null/undefined` в качестве `thisArg` меняют его на `globalThis`. В `strict` – оставляют как есть.

код

// call vs apply – разные форматы аргументов

```
function sum(a, b) { return a + b; }
sum.call(null, 1, 2); // 3
sum.apply(null, [1, 2]); // 3
```

// bind – частичное применение

```
function greet(greeting, name) { return greeting + ', ' + name; }
const hi = greet.bind(null, 'Hi');
hi('Alice'); // 'Hi, Alice'
```

// Заимствование метода

```
function show() {
  const arr = Array.prototype.slice.call(arguments);
  // современнее: const arr = [...arguments] или ...args
}
```

// bind в обработчике – autobind

```
class Btn {
  count = 0;
  constructor() { this.handle = this.handle.bind(this); }
  handle() { this.count++; }
}
```

// Object.prototype.toString – точный type check

```
Object.prototype.toString.call([]); // '[object Array]'
Object.prototype.toString.call(null); // '[object Null]'
Object.prototype.toString.call(/x/); // '[object RegExp]'
```

МОГУТ ДОСПРОСИТЬ

01 Чем `bind` отличается от стрелочной обёртки?

02 Почему `bind` в `JSX-render` считается анти-паттерном для оптимизации?

03 Что вернёт `bound.call(other)` – поменяется ли `this`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/call-apply-decorators>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

2 ВОПРОСА

КЛАССЫ В JS PRIVATE FIELDS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Чем JS class отличается от классических ООП-языков, и как теперь делают приватные поля.

ВОПРОСЫ ДНЯ

Q06 Чем class в JS отличается от class в классических ООП-языках?

Q20 Что такое private fields в class?

Чем class в JS отличается от class в классических ООР-языках?

ОПИСАНИЕ

Class в JS — это синтаксический сахар над функцией-конструктором и прототипным наследованием. Под капотом всё то же, что было до ES2015: `F.prototype`, `new`, `this`, `instanceof`. Но в синтаксисе нет полей доступа в духе `public/private` (без префиксов), нет статической типизации, нет интерфейсов (они в TS), нет перегрузки методов, нет `abstract` как ключевого слова. Зато есть штуки, которых нет в классике: `first-class` функции и классы (можно передавать как аргументы), `expression-форма` (`const C = class { }`), приватные поля `#x`, статические блоки и поля.

КАК РАБОТАЕТ

- 01 `class C { method() {} }` эквивалентен `function C() {} + C.prototype.method = function () {}`, плюс несколько уточнений: метод не `enumerable`, в `strict-режиме` всегда.
- 02 Тело класса в `strict-режиме` всегда. Метод нельзя вызвать без `new`.
- 03 Поля класса (`class fields`, ES2022) — `public x = 1` и приватные `#x = 1` (новый синтаксис, проверяется парсером).
- 04 `Static` — статические поля и методы (на самом классе, не на инстансах).
- 05 `extends` — устанавливает `[[Prototype]]` производного класса к базовому. Доступ к `super.method()` через `[[HomeObject]]`.
- 06 Нет интерфейсов в стандарте JS. В TS — есть, но это `static`.
- 07 Нет перегрузки по типам аргументов — JS однозначно отождествляет имя метода. В TS можно объявить `overload-сигнатуры`, но реализация — одна.
- 08 Нет `abstract` в JS. Конвенция — кидать `NotImplementedError` или `throw` в базовом методе. В TS есть `abstract`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Классы — основа React-классовых компонентов (`legacy`), ORMs, моделей домена, ошибок (`class MyError extends Error`).
- 02 Кастомные ошибки: `class ValidationError extends Error`.
- 03 В Node.js — `Stream`-классы, `EventEmitter`, HTTP-сервер.
- 04 В Web Components — классы с `extends HTMLElement`.

ПОДВОДНЫЕ КАМНИ

- 01 Классы НЕ дают строгую инкапсуляцию по умолчанию. Все методы публичны, кроме новых `private fields` с `#`.
- 02 Стрелочные методы как поля = живут на инстансе, не на `prototype`. У них автоматический `bind`, но они не наследуются и не переопределяются как обычные методы.
- 03 `extends Error` в старых движках работает с багами — `this.constructor` может быть неправильным. В современном коде обычно `Object.setPrototypeOf(this,`

`new.target.prototype`) в конструкторе.

04 `this` в класс-методах – обычный JS `this`, со всеми его проблемами. Подложили метод как колбэк – потеряли `this`.

05 `instanceof` проверяет цепочку прототипов. Если класс загружен в двух копиях (например, в разных бандлах) – `instanceof` может быть `false` для правильного с виду объекта.

КОД

// class – сахар над прототипом

```
class User {
  constructor(name) { this.name = name; }
  greet() { return 'Hi ' + this.name; }
}
// эквивалентно:
function User(name) { this.name = name; }
User.prototype.greet = function () { return 'Hi ' + this.name; };
```

// extends + super

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return this.name + ' speaks'; }
}
class Dog extends Animal {
  speak() { return super.speak() + ' (woof)'; }
}
new Dog('Rex').speak(); // 'Rex speaks (woof)'
```

// static – на самом классе

```
class MathLite {
  static double(x) { return x * 2; }
}
MathLite.double(3); // 6
```

// Кастомная ошибка

```
class ValidationError extends Error {
  constructor(field, msg) {
    super(msg);
    this.name = 'ValidationError';
    this.field = field;
  }
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `[[HomeObject]]` и зачем оно для `super`?
- 02 Почему стрелочные методы в классе не «настоящие» методы?
- 03 Что вернёт `instanceof`, если класс загружен два раза?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/class>

Что такое `private fields` в `class`?

ОПИСАНИЕ

`Private fields` – современный (ES2022) способ объявить приватное поле или метод класса, недоступное снаружи. Имя начинается с `#` и должно быть объявлено в теле класса. Доступ возможен только из методов того же класса. В отличие от старых соглашений (подчёркивание `_foo`) это не «договорённость» – это синтаксическая защита, которую невозможно обойти. До `#fields`-полей приватность достигалась через `WeakMap`, замыкания или `Symbol` – способы рабочие, но многословные.

КАК РАБОТАЕТ

- 01 Объявление: `class C { #x = 1; #log() {} }` – поля и методы должны быть объявлены явно в теле класса.
- 02 Доступ: только из методов того же класса через `this.#x`. Снаружи и из подклассов – `SyntaxError`.
- 03 Проверка наличия: оператор `in` внутри класса: `if (#x in obj) { ... }` – `true`, если `obj` имеет `#x` этого класса.
- 04 Совместимость: если объект не имеет `#x`, обращение к нему выбрасывает `TypeError`. Поэтому `in`-проверка важна для методов, принимающих экземпляр снаружи.
- 05 `Private fields` живут на экземпляре, не на `prototype`.
- 06 Можно использовать со `static`: `static #x` – приватное статическое поле класса.
- 07 Альтернативы до `#fields`: `WeakMap` (наружный «закрытый» стейт), замыкание в конструкторе (методы определены внутри), `Symbol` (не приватно, но «скрыто» от обычных обходов).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Защита внутреннего состояния, которое не должно быть видно снаружи (счётчики, кэши, ссылки на DOM).
- 02 `Domain-models`, где валидация значений только через сеттер/метод, а поле напрямую трогать нельзя.
- 03 Реализация неклонируемых объектов (попытка скопировать через JSON или `Object.assign` – поле не попадёт).
- 04 В библиотеках – для устойчивого API, где внутреннее не должно использоваться клиентами.

ПОДВОДНЫЕ КАМНИ

- 01 Доступ к `#x` по строковому ключу не работает: `this['#x']` – это совершенно другое свойство.
- 02 `TypeError` при обращении к `#x` на чужом объекте – иногда удивляет. Используют `in`-проверку для типобезопасности.

03 Private fields НЕ копируются через `Object.assign` / `spread` / `structuredClone` – это часть инкапсуляции.

04 В наследовании родительские `#fields` НЕ доступны в `child class` – они живут только в своём классе.

05 TS-проекты раньше использовали `TS-private` – это компайл-тайм проверка, на этапе исполнения поле остаётся обычным. Современная рекомендация – `#` для настоящей приватности, `TS-private` – где не нужна `runtime`-защита.

06 Не путать с `Symbol` – `Symbol` не приватный, его можно получить через `Object.getOwnPropertySymbols`.

КОД

// Базовое объявление

```
class Counter {
  #count = 0;
  inc() { this.#count++; }
  get value() { return this.#count; }
}
const c = new Counter();
c.inc();
c.value;    // 1
c.#count;   // SyntaxError
```

// Проверка наличия через `in`

```
class C {
  #x = 1;
  static isOurs(obj) { return #x in obj; }
}
C.isOurs(new C());    // true
C.isOurs({});         // false
```

// Старый способ – `WeakMap`

```
const _state = new WeakMap();
class Counter {
  constructor() { _state.set(this, { count: 0 }); }
  inc() { _state.get(this).count++; }
}
```

// Старый способ – замыкание

```
function Counter() {
  let count = 0;
  this.inc = () => count++;
  this.get = () => count;
}
```

МОГУТ ДОСПРОСИТЬ

01 Что произойдёт при обращении к `#x` на чужом объекте?

02 Почему `Object.assign` не копирует приватные поля?

03 Чем приватные поля отличаются от `Symbol`-ключей?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/private-protected-properties-methods>

PROPERTY DESCRIPTORS**ENUMERABLE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Свойства объекта изнутри: что у них за флаги, и как это всплывает в `for-in` / `Object.keys`.

ВОПРОСЫ ДНЯ

Q07 Что такое property descriptor?

Q08 Чем enumerable property отличается от non-enumerable?

ВОПРОС Q07

Что такое property descriptor?

ОПИСАНИЕ

Property descriptor – это объект-описание свойства объекта, в котором хранятся не только значение, но и флаги: `writable` (можно ли менять), `enumerable` (попадает ли в `for-in` / `Object.keys`), `configurable` (можно ли удалить или переопределить флаги). У свойств-аксессоров вместо `value/writable` – `get/set`. Получить descriptor можно через `Object.getOwnPropertyDescriptor`, изменить – через `Object.defineProperty` или `defineProperties`.

КАК РАБОТАЕТ

- 01 Два типа дескрипторов: `data` (`value`, `writable`) и `accessor` (`get`, `set`). Оба имеют общие `enumerable` и `configurable`.
- 02 Дефолты при `obj.x = 1` (литерал/присваивание): `writable=true`, `enumerable=true`, `configurable=true`.
- 03 Дефолты при `Object.defineProperty` без указания флагов: ВСЕ `false`. Это часто удивляет.
- 04 `writable=false`: попытка записать значение игнорируется в нестрогом, кидает `TypeError` в `strict`.
- 05 `enumerable=false`: свойство не появляется в `for-in` / `Object.keys` / `spread` / `JSON.stringify`.
- 06 `configurable=false`: нельзя удалить и нельзя поменять флаги (кроме перевода `writable` из `true` в `false`).
- 07 `Object.freeze` превращает все дескрипторы в `writable=false`, `configurable=false`.
- 08 Для всех собственных свойств: `Object.getOwnPropertyDescriptors(obj)` – возвращает объект-карту.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Создание `read-only` / `append-only` API: `defineProperty` с `writable=false`.
- 02 Скрытие внутренних полей от перебора: `enumerable=false`.
- 03 Аксессоры (`get/set`) для вычисляемых свойств или валидации.
- 04 Точное копирование объекта со всеми дескрипторами через `getOwnPropertyDescriptors` + `create`.
- 05 Полифиллы и инструментирование – когда нужно подменить свойство, не теряя флагов.

ПОДВОДНЫЕ КАМНИ

- 01 `Object.defineProperty` без флагов делает `enumerable=false` и `writable=false` по умолчанию – частая ошибка новичков.
- 02 `configurable=false` навсегда – back назад нельзя, можно только `writable` из `true` в `false`.

- 03 `Object.assign` копирует только `value`, без флагов. Чтобы сохранить – нужен `defineProperty(s)`.
- 04 `for-in` идёт и по прототипу, и по `enumerable` свойствам обоих.
- 05 `JSON.stringify` пропускает non-enumerable и аксессоры (если нет `toJSON`).
- 06 `Object.keys` в TS не типизирован – возвращает `string[]`, а не `keyof T`. Это TS-нюанс, не JS.

КОД

// Получить дескриптор

```
const obj = { x: 1 };
Object.getOwnPropertyDescriptor(obj, 'x');
// { value: 1, writable: true, enumerable: true, configurable: true }
```

// Скрытое свойство

```
Object.defineProperty(obj, 'hidden', {
  value: 'secret',
  enumerable: false,
});
Object.keys(obj);    // ['x']
obj.hidden;          // 'secret'
```

// Read-only свойство

```
Object.defineProperty(obj, 'CONST', {
  value: 42,
  writable: false,
  configurable: false,
});
obj.CONST = 0;      // молча в нестрог., TypeError в strict
```

// Аксессор – get/set

```
const counter = {
  _n: 0,
  get value() { return this._n; },
  set value(v) { this._n = Math.max(0, v); },
};
counter.value = -5;
counter.value; // 0
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `data-descriptor` отличается от `accessor-descriptor`?
- 02 Что произойдёт при попытке обратно сделать `configurable`?
- 03 Как точно скопировать объект со всеми флагами?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/property-descriptors>

ВОПРОС Q08

Чем enumerable property отличается от non-enumerable?

ОПИСАНИЕ

enumerable – флаг в property descriptor. Если true – свойство попадает в перечисление: for-in, Object.keys, Object.entries, Object.values, JSON.stringify, {...spread}. Если false – свойство существует, доступно по точке/скобкам, но в перечислениях невидимо. По умолчанию литералы создают enumerable свойства, а Object.defineProperty без явного флага – нет. Все методы класса по умолчанию non-enumerable. Это влияет на сериализацию, копирование и видимость для пользовательского кода.

КАК РАБОТАЕТ

- 01 obj.x = 1 → enumerable=true.
- 02 Object.defineProperty(obj, 'x', { value: 1 }) → enumerable=false (это дефолт defineProperty).
- 03 for-in перебирает enumerable свойства самого объекта И его прототипов.
- 04 Object.keys / .values / .entries – только собственные enumerable.
- 05 Object.getOwnPropertyNames – все собственные, включая non-enumerable.
- 06 Spread {...obj} и JSON.stringify – только enumerable.
- 07 Методы класса (class C { f() {} }) и Object.prototype (toString, hasOwnProperty) – все non-enumerable.
- 08 Symbol-ключи отдельно: Object.getOwnPropertySymbols.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Скрытие технических полей от логирования и сериализации.
- 02 Создание immutable read-only API: enumerable=false для внутренних кэшей.
- 03 Полифиллы для встроенных методов – добавляют как non-enumerable, чтобы не сломать чужой for-in.
- 04 Тестовые helpers, которые не должны попадать в snapshot сериализации.

ПОДВОДНЫЕ КАМНИ

- 01 Свойство есть, но Object.keys его не показывает – это enumerable=false. Найти можно через Object.getOwnPropertyDescriptors.
- 02 JSON.stringify теряет non-enumerable свойства. Это иногда сюрприз, когда обычное поле почему-то не сериализуется.
- 03 Object.assign копирует только enumerable. Для полного копирования – getOwnPropertyDescriptors + create.
- 04 Методы класса non-enumerable, поэтому JSON.stringify экземпляра класса не выгрузит методы – это нормально.
- 05 for-in включает прототипные свойства – если расширили Object.prototype, может попасть мусор. Защита – Object.hasOwn.

КОД

// Скрытое поле

```
const obj = { x: 1 };
Object.defineProperty(obj, 'secret', {
  value: 's', enumerable: false,
});
Object.keys(obj);           // ['x']
Object.getOwnPropertyNames(obj); // ['x', 'secret']
obj.secret;                 // 's'
```

// Методы класса – non-enumerable

```
class A { hi() {} }
Object.keys(A.prototype);           // []
Object.getOwnPropertyNames(A.prototype); // ['constructor', 'hi']
```

// JSON.stringify – только enumerable

```
JSON.stringify(obj); // '{"x":1}', secret потерян
```

// Симметричное копирование с дескрипторами

```
const copy = Object.create(
  Object.getPrototypeOf(obj),
  Object.getOwnPropertyDescriptors(obj)
);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что покажет Object.getOwnPropertyNames vs Object.keys?
- 02 Почему методы класса не enumerable?
- 03 Как сделать поле, видимое для for-in, но не для JSON?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/property-descriptors>

ДЕНЬ 5

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

РАВЕНСТВО**COERCION****NUMBER / NaN / Infinity****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Как JS приводит типы и сравнивает значения. Куча edge-cases — спрашивают почти всегда.

ВОПРОСЫ ДНЯ

Q10 Чем == отличается от ===?

Q11 Что такое coercion?

Q12 Какие edge cases есть у Number, NaN, Infinity?

Чем == отличается от ===?

ОПИСАНИЕ

=== – строгое равенство, сравнивает значения только одного типа. Если типы разные – сразу false, без приведения. == – нестрогое равенство, при разных типах пытается привести их к одному (по сложным правилам из спецификации) и сравнить. Поэтому 0 == false, '1' == 1, [] == false и т.д. В современной практике почти всегда пишут ===, потому что приведения у == легко запутать. Единственное исключение – иногда удобно проверить «null или undefined» через x == null.

КАК РАБОТАЕТ

- 01 ===: разные типы – false. Одинаковые типы – обычное сравнение по value (для объектов – по ссылке).
- 02 ==: алгоритм абстрактного равенства из спецификации. Главные правила: number == string → string приводится к number; boolean == X → boolean приводится к number; object == primitive → object приводится к primitive через valueOf/toString; null == undefined.
- 03 NaN === NaN → false, NaN == NaN → false. Object.is(NaN, NaN) → true.
- 04 +0 === -0 → true; Object.is(+0, -0) → false.
- 05 Объекты сравниваются ВСЕГДА по ссылке: {} === {} → false, {} == {} → false.
- 06 x == null истинно только если x === null или x === undefined (полезный shortcut).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 === – почти всегда. ESLint правило eqeqeq включают по умолчанию.
- 02 x == null – короткая проверка «null или undefined».
- 03 В чужом коде встречается == – нужно понимать, что именно сравнивается.
- 04 В сериализованных API – иногда числа приходят как строки, и сравнение === ломается. Правильно – явно конвертировать, а не полагаться на ==.

ПОДВОДНЫЕ КАМНИ

- 01 [] == ![] → true. Лучший пример «не пишите == ».
- 02 '' == 0 → true (оба приводятся к 0). 'abc' == 0 → false ('abc' к числу – NaN, NaN ничему не равно).
- 03 null == undefined → true. null == 0 → false. null < 1 → true. Логика разная, потому что < и == работают по разным алгоритмам.
- 04 [1] == 1 → true (массив приводится к '1' и потом к 1). [1, 2] == '1,2' → true.
- 05 В switch внутри сравнение всегда === (не ==).
- 06 BigInt === Number – всегда false, даже 1n === 1 → false. 1n == 1 → true.

КОД

```
// Без приведений
```

```
1 === '1';    // false
1 == '1';     // true
true == 1;    // true
true === 1;   // false
```

// null/undefined shortcut

```
function defined(x) { return x != null; }
defined(0);           // true
defined(null);        // false
defined(undefined);  // false
```

// NaN – особый случай

```
NaN === NaN;         // false
Number.isNaN(NaN);   // true
Object.is(NaN, NaN); // true
```

// Грабли с массивами

```
[] == ![]; // true ← ![] = false, [] -> 0, false -> 0
[] === ![]; // false
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему [] == ![] возвращает true? Распиши шаги.
- 02 Чем Object.is отличается от ===?
- 03 Когда полезно писать ==?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/comparison>

Что такое coercion?

ОПИСАНИЕ

Coercion (приведение типов) – автоматическое преобразование значения одного типа в другой, которое JS выполняет в арифметических, логических и сравнительных операциях. Бывает явное (`Number(x)`, `String(x)`, `Boolean(x)`) и неявное (`x + ''`, `x * 1`, `!!x`, `+x`). Главные правила coercion знают наизусть: `''` → 0, `'1'` → 1, `'abc'` → NaN, `[]` → `''`, `[a]` → `'a'`, `{}` → `'[object Object]'`, `null` → 0 (через `Number`), `undefined` → NaN, `true/false` → 1/0.

КАК РАБОТАЕТ

- 01 Бинарный + при наличии строки конкатенирует, иначе складывает: `1 + '2' === '12'`; `1 + 2 === 3`.
- 02 Унарный `+x` → `Number(x)`. `+''` === 0, `+'abc'` === NaN.
- 03 `Object` → `primitive`: вызывается `Symbol.toPrimitive`, потом `valueOf`, потом `toString`. Hint зависит от контекста (`string/number/default`).
- 04 В `if/while/||/&&` значения приводятся к `Boolean`. Falsy: `false`, 0, -0, 0n, `''`, `null`, `undefined`, NaN. Все остальные – `truthy` (включая `[]` и `{}`).
- 05 В сравнении `<` `>` `<=` `>=` оба операнда приводятся к `Number` (или к строкам, если оба строки – лексикографически).
- 06 `Boolean('false')` → `true` (строка непустая). Это часто удивляет.
- 07 `Boolean({})` → `true`; `Boolean([])` → `true`. Объекты `truthy` независимо от содержимого.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Boolean(x)` – явная проверка на `truthy/falsy`.
- 02 `Number(x)` – преобразование строки в число с проверкой (NaN, если не валидно).
- 03 `String(x)` – безопасное преобразование, даже `null/undefined`.
- 04 `Symbol.toPrimitive` – для пользовательских преобразований (`Date`, `BigInt`).
- 05 Сравнения и арифметика повсеместно – coercion работает автоматически, даже когда не хочется.

ПОДВОДНЫЕ КАМНИ

- 01 `[] + []` === `''` (массивы становятся `''`), `[] + {}` === `'[object Object]'`, `{}` + `[]` (как выражение) → 0 (зависит от парсинга).
- 02 `'5' - 1` === 4 (минус приводит к `Number`), `'5' + 1` === `'51'` (плюс конкатенирует со строкой).
- 03 `Number('1.5e2')` === 150 (научная нотация работает); `Number('1,000')` === NaN (запятая не цифра).
- 04 `Symbol` → `Number` – `TypeError`. `Symbol` → `String` возможно только через `String(s)`, не `+s`.

05 `Boolean('false') === true` – потому что строка непустая. Для парсинга булевых лучше явно: `x === 'true'`.

06 `JSON.stringify(undefined) === undefined` (не строка). `JSON.stringify(null) === 'null'`.

КОД

// Унарные операторы

```
+'';      // 0
+'5';     // 5
+'abc';   // NaN
!!'a';    // true (truthy → true)
!'';      // false
!!0;      // false
!!{};     // true
```

// Плюс с разными типами

```
1 + '2';   // '12'
'5' - 1;   // 4
true + true; // 2
[] + 1;     // '1'
[1] + [2];  // '12'
```

// Boolean truthy/falsy

```
Boolean(0);      // false
Boolean('');     // false
Boolean('false'); // true (непустая строка!)
Boolean({});     // true
Boolean([]);     // true
```

// Свой toPrimitive

```
const t = {
  [Symbol.toPrimitive](hint) {
    return hint === 'string' ? 'X' : 1;
  },
};
+t;      // 1
`${t}`;  // 'X'
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие 7 значений в JS falsy?
- 02 Что вернёт `{}` + `[]` и почему ответ зависит от контекста?
- 03 Чем `String(x)` отличается от `''+x`?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/type-conversions>

Какие edge cases есть у Number, NaN, Infinity?

ОПИСАНИЕ

Number в JS — это IEEE 754 double, единственный числовой тип помимо BigInt. Из этого вытекают все сюрпризы: $0.1 + 0.2 \neq 0.3$, целые числа точны только до $2^{53} - 1$ (Number.MAX_SAFE_INTEGER), деление на ноль даёт Infinity, а не ошибку, NaN не равен сам себе, +0 и -0 различаются по Object.is, parseInt и Number парсят по-разному. Знание этих углов спасает от багов в денежных и идентификаторных вычислениях.

КАК РАБОТАЕТ

- 01 `0.1 + 0.2 === 0.30000000000000004`. Это IEEE 754 — не баг JS, а свойство формата. Для денежных — целые копейки или библиотеки (decimal.js, big.js).
- 02 Безопасные целые: до $2^{53} - 1$ (Number.MAX_SAFE_INTEGER = 9007199254740991). Дальше точность теряется. Большие ID API часто приходят строками.
- 03 Number.isInteger / Number.isSafeInteger / Number.isFinite / Number.isNaN — строгие проверки, не приводят аргумент.
- 04 `isNaN(x)` (глобальный) сначала приводит x к числу — `isNaN('abc') === true`. Number.isNaN — без приведения.
- 05 Деление: `1/0 === Infinity`; `-1/0 === -Infinity`; `0/0 === NaN`.
- 06 `+0 === -0 (true)`, но `Object.is(+0, -0) === false`. `1/+0 === Infinity`, `1/-0 === -Infinity`.
- 07 `parseInt('10px') === 10`, `Number('10px') === NaN`.
- 08 `parseInt('0x10') === 16`, `parseInt('010')` в современном JS — 10 (раньше было 8, восьмеричное).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые финансовые расчёты — никогда не доверять Number напрямую. Хранить в копейках или decimal-либах.
- 02 Большие ID — хранить строкой, сравнивать строкой.
- 03 Конверсия пользовательского ввода — Number с проверкой isFinite + isNaN.
- 04 Парсинг разнородных данных — parseInt/parseFloat для «взять число из начала строки».

ПОДВОДНЫЕ КАМНИ

- 01 `0.1 + 0.2 !== 0.3` — самый известный пример. Лечится `Math.abs(a - b) < 1e-9` или библиотекой.
- 02 `NaN === NaN === false`. Проверка через Number.isNaN или Object.is.
- 03 Number.MAX_VALUE и Number.MIN_VALUE — не максимум и минимум, как кажется. MIN_VALUE — самое маленькое положительное ($\approx 5e-324$).

- 04 `Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2` → `true` (потеря точности).
- 05 `parseInt('123abc', 10) === 123` – не падает, а берёт префикс. Иногда полезно, иногда – баг.
- 06 `Number(' 123 ')` === 123 (пробелы), `Number('')` === 0 (пустая строка!). Часто сюрприз.
- 07 `BigInt` и `Number` нельзя смешивать в одной операции – `TypeError`. Нужно явно: `Number(big)` или `BigInt(num)`.

КОД

// Точность

```
0.1 + 0.2;           // 0.30000000000000004
(0.1 + 0.2).toFixed(2); // '0.30'
Math.abs(0.1 + 0.2 - 0.3) < 1e-9; // true
```

// Безопасные целые

```
Number.MAX_SAFE_INTEGER; // 9007199254740991
9007199254740993 === 9007199254740992; // true (потеря)
```

// NaN

```
NaN === NaN;           // false
Number.isNaN(NaN);     // true
isNaN('abc');           // true (с приведением)
Number.isNaN('abc');   // false
```

// Infinity и деление

```
1 / 0;           // Infinity
-1 / 0;          // -Infinity
0 / 0;           // NaN
Number.isFinite(Infinity); // false
```

// parseInt vs Number

```
Number('10px'); // NaN
parseInt('10px'); // 10
Number('');      // 0
parseInt('');     // NaN
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `Number.isNaN` отличается от глобального `isNaN`?
- 02 Как корректно сравнивать дробные числа?
- 03 Какой тип использовать для денег?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/number>

OPTIONAL CHAINING NULLISH || vs ??**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Современные операторы — синтаксический сахар, но детали важны (?? только null/undefined, не falsy).

ВОПРОСЫ ДНЯ

- Q13 Что такое optional chaining?
- Q14 Что такое nullish coalescing?
- Q15 Чем || отличается от ??

ВОПРОС Q13

Что такое optional chaining?

ОПИСАНИЕ

Optional chaining (?.) – оператор, который безопасно обращается к свойству или вызывает метод. Если перед ?. стоит null или undefined – выражение сразу возвращает undefined без ошибки. Поддерживает три формы: a?.b (свойство), a?.[b] (вычисляемое), a?.() (вызов функции). Это сокращает многословные if-проверки: вместо a && a.b && a.b.c пишут a?.b?.c. Появилось в ES2020, активно используется в TypeScript для работы с возможно-undefined полями.

КАК РАБОТАЕТ

- 01 a?.b: если a === null || a === undefined → результат undefined. Иначе – обычный a.b.
- 02 a?.[expr]: то же для bracket-доступа. Полезно для динамических ключей.
- 03 a?.(): то же для вызова. Полезно для опциональных колбэков.
- 04 Цепочка короткозамкнута: при первом null/undefined оставшаяся часть НЕ выполняется. a?.b().c().d – если a undefined, .b() даже не вызывается.
- 05 Не путать с x?.y = 1 – это синтаксическая ошибка. Optional chaining не работает в LHS присваивания.
- 06 Не одно и то же с !! – ?. возвращает undefined, не false. Часто комбинируется с ?? для дефолта.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Доступ к глубоко вложенным полям API-ответа: data?.user?.address?.city.
- 02 Опциональные колбэки: onChange?.(e).
- 03 Динамические ключи: config?.[key].
- 04 Безопасное обращение к DOM: document.querySelector('.x')?.click().
- 05 В TS-коде с union-типами и опциональными полями – почти обязательный синтаксис.

ПОДВОДНЫЕ КАМНИ

- 01 Только null/undefined короткозамыкают. Другие falsy (0, '', false) – НЕ. a?.b где a = 0 → TypeError (нельзя читать .b у 0).
- 02 ?. возвращает undefined, не дефолт. Чтобы дефолт – ?. в паре с ??: a?.b ?? defaultValue.
- 03 В цикле/мапе можно «съесть ошибку» там, где надо было явно её обработать. Чрезмерное ?. маскирует баги.
- 04 ?.() и обычный () – разная семантика. obj?.fn?.() – сначала obj, потом проверка fn, потом вызов.
- 05 Не работает в LHS: x?.y = 1 → SyntaxError.

06 Долго отсутствовало в Internet Explorer – но IE уже практически мёртв. Современные таргеты – норма.

код

// Доступ к глубокому полю

```
const city = data?.user?.address?.city;  
// undefined, если любое звено отсутствует
```

// Bracket и вызов

```
const v = config?.[key];      // вычисляемый ключ  
onChange?.(value);           // опциональный колбэк
```

// Только null/undefined короткозамыкают

```
const a = 0;  
a?.x;           // TypeError? – нет, undefined  
// потому что 0 != null/undefined  
// 0?.toString() === '0' (метод вызывается)
```

// Связка с ??

```
const name = user?.name ?? 'guest';
```

// Не работает в LHS

```
obj?.x = 1; // SyntaxError
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт 0?.toString()?
- 02 Почему ?. возвращает undefined, а не дефолт?
- 03 Чем отличается a?.b от a && a.b?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/optional-chaining>

ВОПРОС Q14

Что такое nullish coalescing?

ОПИСАНИЕ

Nullish coalescing (??) – оператор, который возвращает правый операнд только если левый равен null или undefined. Все остальные значения, включая 0, '', false, NaN, проходят как «нормальные» и возвращаются как есть. В отличие от ||, который реагирует на любые falsy, ?? стреляет только по двум значениям. Появился в ES2020, часто используется вместе с ?. для проставления дефолтов.

КАК РАБОТАЕТ

- 01 a ?? b: если a === null || a === undefined → возвращает b. Иначе – a.
- 02 Короткое замыкание: b вычисляется только если a – null/undefined.
- 03 Нельзя смешивать с || и && без скобок: a || b ?? c – SyntaxError. Нужно (a || b) ?? c или a || (b ?? c).
- 04 В отличие от || не реагирует на 0, '', false, NaN.
- 05 Часто пара с optional chaining: obj?.prop ?? default.
- 06 В TS – стандартный способ задать дефолт для опциональных полей.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дефолт для конфигов: const port = config.port ?? 3000.
- 02 Дефолт для опциональных props в React: const size = props.size ?? 'md'.
- 03 Безопасный доступ к API-ответу: const items = data?.items ?? [].
- 04 Замена || там, где falsy-нюансы важны (0, '' должны пройти как валидные значения).

ПОДВОДНЫЕ КАМНИ

- 01 Без скобок с ||/&& – SyntaxError. Это намеренная защита от двусмысленности.
- 02 ?? не реагирует на NaN – NaN ?? x === NaN. Иногда удивляет.
- 03 Если хочется и null, и пустую строку считать невалидным – ?? не подходит, нужен ||.
- 04 Полифилл/transpile в старые ES – bundler'ы это умеют, но проверять. Большой код может разрастись.
- 05 В типе TS string | undefined ?? '' даёт string. С string | null | undefined тоже string. Это удобно, но можно случайно потерять полезное значение.

КОД

// Базовое использование

```
const x = value ?? 'default';  
// 'default' только если value === null/undefined
```

// Сравнение с ||

```
0 ?? 5;    // 0
0 || 5;    // 5
'' ?? 'd'; // ''
'' || 'd'; // 'd'
```

// Связка с ?.

```
const city = data?.user?.address?.city ?? 'unknown';
```

// Скобки обязательны при смеси

```
// SyntaxError:
// const v = a || b ?? c;
// Корректно:
const v = (a || b) ?? c;
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему ?? и || без скобок – SyntaxError?
- 02 Что вернёт NaN ?? 'd'?
- 03 Когда выбрать ||, а когда ??

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/nullish-coalescing-operator>

ВОПРОС Q15

Чем || отличается от ??

ОПИСАНИЕ

|| возвращает правый операнд при любом falsy левом: false, 0, -0, '', null, undefined, NaN. ?? – только при null или undefined. Поэтому 0 || 5 === 5, а 0 ?? 5 === 0. Это критично для дефолтов: 0 – валидное число, '' – валидная пустая строка, и сбросить их в дефолт через || обычно ошибка. ?? – современный, точный способ задать дефолт «только если ничего нет», || – старый и более размашистый.

КАК РАБОТАЕТ

- 01 || – логическое ИЛИ. Возвращает первый truthy операнд или последний falsy.
- 02 ?? – nullish coalescing. Возвращает правый только при null/undefined.
- 03 Falsy для ||: false, 0, -0, 0n, '', null, undefined, NaN.
- 04 Nullish для ??: null, undefined.
- 05 Оба коротко-замкнуты: правый не вычисляется, если левый уже подходит.
- 06 Запрещено смешивать без скобок: a || b ?? c – SyntaxError.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 ?? – современный дефолт для опциональных значений: props.count ?? 0, env.PORT ?? 3000.
- 02 || – когда любое falsy должно подменяться (например, user.displayName || 'Anonymous' – пустая строка тоже становится 'Anonymous').
- 03 В legacy-коде || для дефолтов встречается часто. При рефакторинге решать индивидуально, нужно ли менять на ??.

ПОДВОДНЫЕ КАМНИ

- 01 0 и '' с || превращаются в дефолт – самая частая ловушка. Для счётчиков и инпутов почти всегда нужно ??.
- 02 false с || → дефолт. Для boolean-флагов это редко правильно.
- 03 ?? и || визуально похожи в коде – легко перепутать. Code review и линтеры (no-mixed-operators) помогают.
- 04 || с левым truthy НЕ вычисляет правый – иногда полезно (short-circuit evaluation), иногда забывают про побочный эффект.
- 05 Без скобок при смешении || и ?? в одной выражении – SyntaxError. Это намеренно.

КОД

```
// Falsy ловушки с ||
const count = props.count || 10; // если props.count===0 → 10
const count2 = props.count ?? 10; // 0 проходит как 0

// false с ||
```

```
const visible = props.visible || true; // false → true!  
const visible2 = props.visible ?? true; // false проходит
```

// Когда || ок

```
const name = user.displayName || user.email || 'Anon';  
// тут '' тоже невалидно, || ок
```

// Скобки при смеси

```
const v = (a || b) ?? c;
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие 7 значений в JS falsy?
- 02 Когда || лучше, чем ??
- 03 Что вернёт null || 0 ?? 'd'?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/nullish-coalescing-operator>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

4 ВОПРОСА

ITERABLE / ITERATOR**GENERATORS****SYMBOLS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Закрепление: как устроены for-of, генераторы, что такое Symbol и где он реально нужен.

ВОПРОСЫ ДНЯ

Q16 Что такое iterable и iterator?

Q17 Как работает generator?

Q18 Что такое symbol?

Q19 Где используются symbols?

Что такое iterable и iterator?

ОПИСАНИЕ

Iterable – объект, у которого есть метод по ключу `Symbol.iterator`, возвращающий `iterator`. `Iterator` – объект с методом `.next()`, который возвращает `{ value, done }`. Когда `done = true`, итерация закончена. На этом протоколе работают `for-of`, `spread (...)`, `destructuring` массивов, `Promise.all`, конструкторы `Map/Set`, `Array.from`. Встроенные итераторы: `Array`, `String`, `Map`, `Set`, `NodeList`, `arguments`, `generator-функции`.

КАК РАБОТАЕТ

- 01 `Iterable: { [Symbol.iterator]() { return iterator; } }`.
- 02 `Iterator: { next() { return { value, done } }, опц. return(), throw() }`.
- 03 `for-of` (и `spread`, и `destructuring`) ищет `Symbol.iterator`, вызывает его, потом цикл `next()`.
- 04 После `done=true` итератор «исчерпан» – повторный `next()` обычно тоже возвращает `done=true`.
- 05 `Iterable` можно сделать самому – реализовать `Symbol.iterator`. Часто проще – генератор.
- 06 `Async iterator` – `Symbol.asyncIterator + async next() → Promise<{value, done}>`. Используется в `for-await-of`.
- 07 `Iterators` сами по себе тоже `iterable` (`next`-метод + `Symbol.iterator` возвращает себя).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `for-of` по массивам, строкам, мапам, сетам.
- 02 `Spread (...)` – копирование, конкатенация массивов.
- 03 `Destructuring [a, b, c] = iterable`.
- 04 `Array.from(iterable)` – превращение в массив.
- 05 `Map/Set` ручной обход через `.entries()` / `.keys()` / `.values()` – все возвращают итераторы.
- 06 `Custom data structures` – реализовать `iterable`, чтобы пользователь мог писать `for-of` по объекту.

ПОДВОДНЫЕ КАМНИ

- 01 `for-in` vs `for-of`: `for-in` – по ключам объекта (включая наследованные); `for-of` – по значениям `iterable`.
- 02 После исчерпания итератор не «перематывается» – нужно вызвать `Symbol.iterator` заново.
- 03 `Plain object {}` НЕ `iterable`. `for-of` по нему – `TypeError`.
- 04 `Object.entries(obj)` – возвращает массив, но массив итерируем – поэтому работает.

05 Iterator с побочными эффектами – повторное чтение может приводить к разным результатам.

КОД

// for-of по строке

```
for (const ch of 'abc') console.log(ch);  
// 'a', 'b', 'c'
```

// Свой iterable

```
const range = {  
  from: 1, to: 3,  
  [Symbol.iterator]() {  
    let i = this.from;  
    const end = this.to;  
    return {  
      next() {  
        return i <= end  
          ? { value: i++, done: false }  
          : { value: undefined, done: true };  
      },  
    };  
  },  
};  
[...range]; // [1, 2, 3]
```

// Через генератор – короче

```
const range = {  
  *[Symbol.iterator]() {  
    for (let i = 1; i <= 3; i++) yield i;  
  },  
};
```

// Async iteration

```
async function* lines(stream) {  
  for await (const chunk of stream) yield chunk.toString();  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем for-in отличается от for-of?
- 02 Можно ли сделать iterable из обычного {} объекта?
- 03 Что вернёт next() после исчерпания итератора?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/iterable>

Как работает generator?

ОПИСАНИЕ

Generator — это функция, объявленная через `function*`, которая может приостанавливать своё выполнение через `yield` и возобновлять при следующем вызове `.next()`. Генератор возвращает `iterator` (на самом деле `iterable` + `iterator` одновременно), который проходится через `for-of` или вручную. `yield expr` — отдаёт значение наружу, `next(arg)` — передаёт значение обратно внутрь как результат `yield`. Генераторы используются для ленивых последовательностей, кастомных `iterable`, `async-control-flow` до `async/await`.

КАК РАБОТАЕТ

- 01 `function* fn() { yield x; }` — объявление генератора.
- 02 Вызов `fn()` возвращает генератор-объект, не выполняет тело.
- 03 `gen.next()` — запускает/продолжает до следующего `yield`. Возвращает `{ value: yielded, done: false }` или `{ value: returned, done: true }` после `return` / окончания.
- 04 `yield expr` — приостанавливает; `expr` становится `value` в `next()`-результате.
- 05 Значение, переданное в `next(arg)`, становится результатом выражения `yield` внутри генератора (важно для двусторонней коммуникации).
- 06 `yield* delegate` — делегирует другому `iterable`: пример композиции генераторов.
- 07 `gen.return(v)` — досрочно завершает; `gen.throw(e)` — выбрасывает ошибку внутрь.
- 08 `Async generator (async function*)` + `for-await-of` — для асинхронных последовательностей.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Ленивые последовательности — числа, страницы пагинации, бесконечные потоки.
- 02 Кастомные `iterable` — короткая запись через `function*`.
- 03 Композиция: `yield* generator()` для объединения последовательностей.
- 04 `Async generators` — потоки данных (`chunks` из `ReadableStream`, лог-файлы).
- 05 Раньше — `control-flow` для `async`-кода (`co.js`, `redux-saga`). Сейчас вытеснено `async/await`, но `redux-saga` до сих пор живёт.

ПОДВОДНЫЕ КАМНИ

- 01 Генератор-функция и обычная — разные. `function*` нельзя использовать как обычный конструктор без подвохов.
- 02 После `return`/исчерпания все `next()` возвращают `{value:undefined, done:true}`.
- 03 `next(arg)` — первый вызов `arg` игнорируется (нет ещё `yield`, который мог бы его принять).
- 04 В `async`-генераторе `yield` ждёт промис автоматически (`yield Promise` → `next()` вернёт `Promise<{value, done}>`).
- 05 Дебаггинг генераторов сложнее — стек вызовов нелинейный.

06 Генератор хранит замкнутое состояние, как замыкание. Если он попал в долгоживущее место — память может удерживаться.

КОД

// Базовый генератор

```
function* counter() {
  let i = 0;
  while (true) yield i++;
}
const gen = counter();
gen.next(); // { value: 0, done: false }
gen.next(); // { value: 1, done: false }
```

// for-of по генератору

```
function* range(n) {
  for (let i = 0; i < n; i++) yield i;
}
for (const x of range(3)) console.log(x);
// 0, 1, 2
```

// yield* делегирование

```
function* a() { yield 1; yield 2; }
function* b() { yield 0; yield* a(); yield 3; }
[...b()]; // [0, 1, 2, 3]
```

// Двусторонняя коммуникация

```
function* talk() {
  const name = yield 'who?';
  yield 'hi ' + name;
}
const g = talk();
g.next(); // { value: 'who?', done: false }
g.next('Alice'); // { value: 'hi Alice', done: false }
g.next(); // { value: undefined, done: true }
```

// Async generator

```
async function* lines(reader) {
  while (true) {
    const { value, done } = await reader.read();
    if (done) return;
    yield value.toString();
  }
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вернёт первый next(arg)? Куда уходит arg?
- 02 Чем отличается yield* от обычного yield?
- 03 Когда использовать generator вместо async/await?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/generators>

Что такое symbol?

ОПИСАНИЕ

`Symbol` – примитивный тип, представляющий уникальный идентификатор. Создаётся через `Symbol(description)`. Каждый вызов `Symbol()` возвращает новое значение, не равное никакому другому. `Symbol` используется как ключ свойства, который не конфликтует с обычными строками – так делают «не-публичные» поля и расширения встроенных протоколов (`Symbol.iterator`, `Symbol.toPrimitive`). Уникальность символа делает его удобным для маркеров и метаданных.

КАК РАБОТАЕТ

- 01 `Symbol()` – каждый раз новый. `Symbol() === Symbol()` → `false`.
- 02 `Symbol(description)` – `description` только для отладки, не влияет на идентичность.
- 03 `Symbol`-ключи в объекте: `const KEY = Symbol(); obj[KEY] = 1`. Доступ только по этому символу.
- 04 `Symbol.for(key)` – глобальный реестр: возвращает существующий или регистрирует новый по строке-ключу. `Symbol.for('x') === Symbol.for('x')` → `true`.
- 05 `Symbol`-ключи не попадают в `for-in` / `Object.keys` / `JSON.stringify`. Получить можно через `Object.getOwnPropertySymbols`.
- 06 Well-known symbols – стандартные точки расширения: `Symbol.iterator`, `Symbol.asyncIterator`, `Symbol.toPrimitive`, `Symbol.hasInstance`, `Symbol.toStringTag`.
- 07 `typeof Symbol()` === `'symbol'`.
- 08 `Symbol` нельзя автоматически привести к строке через `+`. Только `String(s)`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Уникальные ключи свойств для библиотек (избежать конфликта с пользовательскими полями).
- 02 Реализация `iterable` (`Symbol.iterator`), кастомного приведения к примитиву (`Symbol.toPrimitive`), `instanceof` (`Symbol.hasInstance`).
- 03 Маркеры для `type discrimination` в `reducers/state-machines`: `const TAG = Symbol('Tag')`.
- 04 Расширение объекта без боязни сломать чужой код, опирающийся на `for-in`.

ПОДВОДНЫЕ КАМНИ

- 01 `Symbol`-ключи не сериализуются в `JSON`. Если хранить состояние через `Symbol` – потеря после `JSON.parse/stringify`.
- 02 `Symbol()` и `Symbol()` – разные. `Symbol('x')` и `Symbol('x')` – разные. Только `Symbol.for('x') === Symbol.for('x')`.
- 03 Нельзя использовать `Symbol` как ключ в `WeakMap` (только объекты). В новых пропозиалах это меняется, но в текущем стандарте нельзя.

- 04 Auto-coerce: + Symbol() → TypeError. `\${s}` → TypeError. Нужно String(s).
- 05 Symbol-ключи проходят через Object.assign (копируются), это иногда сюрприз.
- 06 TS: type symbol – обычно symbol, но для ключевого использования часто нужен unique symbol.

КОД

// Уникальные символы

```
const a = Symbol('id');
const b = Symbol('id');
a === b; // false
Symbol.for('x') === Symbol.for('x'); // true (registry)
```

// Symbol как ключ – скрыт от for-in

```
const KEY = Symbol('hidden');
const obj = { x: 1, [KEY]: 'secret' };
Object.keys(obj); // ['x']
Object.getOwnPropertySymbols(obj); // [Symbol(hidden)]
```

// Well-known: Symbol.iterator

```
const range = {
  *[Symbol.iterator]() { yield 1; yield 2; }
};
[...range]; // [1, 2]
```

// Symbol.toPrimitive

```
const t = {
  [Symbol.toPrimitive](hint) {
    return hint === 'string' ? 'X' : 1;
  }
};
+t; // 1
`${t}`; // 'X'
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем Symbol() отличается от Symbol.for()?
- 02 Почему Symbol-ключи не попадают в JSON?
- 03 Что делает Symbol.toStringTag?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/symbol>

Где используются symbols?

ОПИСАНИЕ

`Symbol` – узкий, но важный инструмент. Главные сценарии: реализация `iterable` через `Symbol.iterator` (что лежит в основе `for-of`, `spread`, `destructuring`); кастомизация приведения через `Symbol.toPrimitive` и `Symbol.toStringTag`; создание уникальных «не-конфликтных» ключей в объектах (для библиотек и фреймворков); `type discrimination` через уникальные маркеры в `action-types` и `state-machines`; глобальные «волшебные» точки в стандарте – `well-known symbols` (`Symbol.hasInstance`, `Symbol.species`).

КАК РАБОТАЕТ

- 01 `Symbol.iterator` – точка входа `for-of`, `spread`, `destructuring` массивов.
- 02 `Symbol.asyncIterator` – `async`-аналог, используется в `for-await-of`.
- 03 `Symbol.toPrimitive` – управляет авто-преобразованием объекта в примитив.
- 04 `Symbol.toStringTag` – управляет результатом `Object.prototype.toString.call(obj)` (`'[object Tag]'`).
- 05 `Symbol.hasInstance` – управляет поведением `instanceof`.
- 06 `Symbol.species` – позволяет подклассу влиять на тип, возвращаемый встроенными методами (`Array.prototype.map` и т.д.).
- 07 `Symbol.for` / `Symbol.keyFor` – глобальный `registry` для разделения значений между модулями.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кастомные коллекции – реализуйте `Symbol.iterator`, и `for-of` будет работать.
- 02 Дата-классы – `Symbol.toPrimitive` для `arithmetic`, `Symbol.toStringTag` для отладочного вывода.
- 03 `Type discrimination` в `state-machines`: `const STARTED = Symbol('STARTED');` `switch (event.type) { case STARTED: ... }`.
- 04 Библиотеки (`React`, `Redux`, `Svelte`) часто используют `Symbol` для внутренних маркеров – невозможно случайно пересечь.
- 05 Имитация приватных полей (старый паттерн до `#fields`) – ключ-`Symbol` хранится в замыкании.

ПОДВОДНЫЕ КАМНИ

- 01 `Symbol`-ключи не попадают в `JSON`, не копируются `JSON.parse/stringify`.
- 02 Объект с `Symbol`-ключом может выглядеть «пустым» через `Object.keys`, что иногда мешает дебагу.
- 03 `Symbol()` $\neq$ `Symbol()` – для разделения между модулями нужен `Symbol.for`.
- 04 Все `well-known symbols` – это уже значения в `Symbol`-объекте: `Symbol.iterator`, не строка. Иногда путают, ставят строку, получают баг.

05 TS требует unique symbol для использования в качестве key в типе. Обычный symbol – не подойдёт.

КОД

// iterable через Symbol.iterator

```
class Range {
  constructor(n) { this.n = n; }
  *[Symbol.iterator]() {
    for (let i = 0; i < this.n; i++) yield i;
  }
}
[...new Range(3)]; // [0, 1, 2]
```

// Symbol.toStringTag

```
class Money {
  get [Symbol.toStringTag]() { return 'Money'; }
}
Object.prototype.toString.call(new Money());
// '[object Money]'
```

// Type discrimination

```
const ACTION_LOGIN = Symbol('login');
const ACTION_LOGOUT = Symbol('logout');
switch (action.type) {
  case ACTION_LOGIN: ...
}
```

// Реестр через Symbol.for

```
// в одном модуле
const KEY = Symbol.for('app/state');
// в другом
const SAME = Symbol.for('app/state');
KEY === SAME; // true
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Symbol.species и где встречается?
- 02 Почему библиотеки используют Symbol, а не строковые ключи?
- 03 Когда Symbol удобнее, чем приватные поля #?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/symbol>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Объяснить prototype chain и нарисовать цепочку для {} и [] (Q1)
- ☐ Описать, что делает new по шагам (Q5)
- ☐ Сравнить Object.create / assign / freeze (Q9)
- ☐ Объяснить this в 5 ситуациях вызова (Q2)
- ☐ Сказать, чем стрелка отличается от обычной функции (Q3)
- ☐ Расписать call / apply / bind с разницей в формате аргументов (Q4)
- ☐ Описать, чем JS class отличается от классических ООП-языков (Q6)
- ☐ Рассказать о приватных полях # и сравнить с WeakMap-паттерном (Q20)
- ☐ Описать property descriptor и его флаги (Q7, Q8)
- ☐ Распутать пазл с == и ===, объяснить [] == ![] (Q10, Q11)
- ☐ Назвать минимум 3 edge case у Number/NaN/Infinity (Q12)
- ☐ Сравнить optional chaining и старый && a && a.b (Q13)
- ☐ Сравнить ?? и ||, объяснить когда выбрать какое (Q14, Q15)
- ☐ Описать iterable + iterator и реализовать свой через generator (Q16, Q17)
- ☐ Сказать, что такое Symbol и привести 2 случая использования (Q18, Q19)